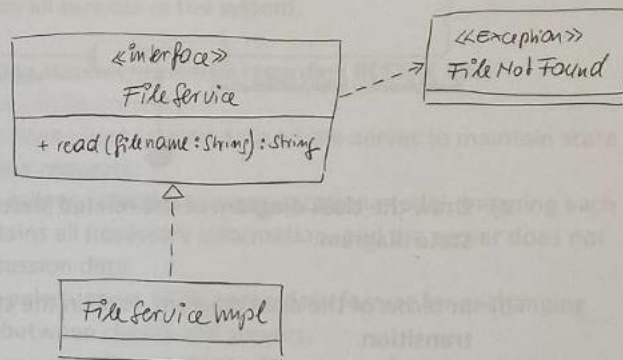


Exam Questions

answer 26.02.2025

1) Given the following class diagram: [4+ 3 + 3 points]



- In this **class diagram**, add a **Validation Proxy** and the "Client".
- Draw the **object diagram** from the "Client" to the instance of the implementation class.
- Draw the **sequence diagram** for the "Client", the Validation Proxy and the instance of the implementation class.

Pay attention to the UML notation!

1) Proxy (Validation Proxy)

1(a) Extend the class diagram with a Validation Proxy and the Client

To apply the **Proxy pattern**, we introduce a class **ValidationProxy** that implements the same interface as the real service.

- The given interface is **FileService** with the method `read(pathname: string): string`.
- **FileServiceImpl** is the real implementation of FileService (the real subject).
- Add a **Client** class that only depends on the interface FileService.
- Add **ValidationProxy** which also implements FileService and contains a reference to the real implementation (FileServiceImpl).

Relationships to show in the class diagram (written description):

- FileService is an interface.
- FileServiceImpl implements FileService.
- ValidationProxy implements FileService.
- ValidationProxy has a member/reference like `realService: FileService (or FileServiceImpl)`.
- Client uses FileService, but in reality it will be connected to the ValidationProxy.

Purpose: The proxy checks the pathname (validation) before delegating the call to the real service.

SDA_2025_26_02_Final__Exam

1(b) Object diagram (instances and references)

In the object diagram we show real objects (instances):

- One instance of **Client**.
- One instance of **ValidationProxy**.
- One instance of **FileServiceImpl**.

Connections:

- The Client holds a reference to FileService, which points to the ValidationProxy instance.
- The ValidationProxy instance holds a reference to the real FileServiceImpl instance.

This shows the client is not calling the real implementation directly; it goes through the proxy.

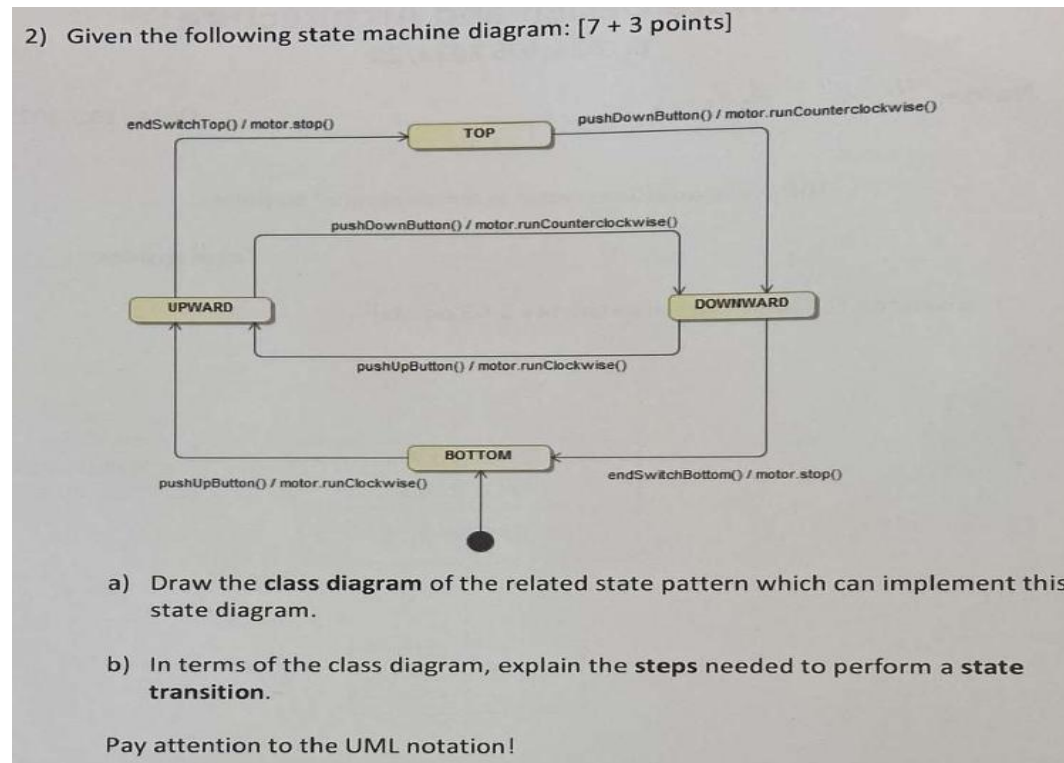
SDA_2025_26_02_Final__Exam

1(c) Sequence diagram (processing of `Client.read("abc.txt")`)

The sequence is:

1. The **Client** calls `read("abc.txt")` on the **ValidationProxy** (typed as `FileService`).
2. The **ValidationProxy** performs validation (for example: check that pathname is not empty, has valid format, allowed path etc.).
3. If validation passes, the **ValidationProxy** forwards the call to the **FileServiceImpl** by calling `read("abc.txt")`.
4. The **FileServiceImpl** returns the file content (string) back to the **ValidationProxy**.
5. The **ValidationProxy** returns the same content to the **Client**.

If the pathname is invalid or the file does not exist, a **FileNotFoundException** exception is thrown (either by proxy or by the real service and passed through).



2) State machine → State Pattern

2(a) Class diagram of the State Pattern for the given state machine

To model the state machine as a **State Pattern**, we create:

- A **Context** class (example name: Controller) that holds the current state.
- A **State** interface (or abstract class) that declares the event methods (the triggers).
- Concrete State classes for each state in the diagram.

Required elements (written):

- Controller (Context) contains a field `state: State`.
- State defines methods corresponding to the events, for example:
 - `pushUpButton()`, `pushDownButton()`, `endSwitchTop()`, `endSwitchBottom()`.
- Concrete state classes:
 - `TopState`, `UpwardState`, `DownwardState`, `BottomState` (one class per state in the diagram).
- The Controller also has access to a `Motor` object to execute actions:
 - `runClockwise`, `runCounterclockwise`, `stop`.

The Controller delegates each event call to the currently active State object.

SDA_2025_26_02_Final__Exam

2(b) Steps needed to perform a state transition

A state transition happens in these steps:

1. An external event occurs (e.g., the user presses Up/Down or an end switch is reached).
2. The Context (Controller) receives the event call.
3. The Controller forwards the event to the current State object.
4. The current State object decides:
 - which motor action to execute (run/stop direction),
 - and whether a state change is needed.
5. If a transition is required, the State object asks the Controller to change its state (for example: `Controller.setState(new UpwardState())`).
6. After this, the next events are handled by the new State object.

This avoids long if/else statements and keeps behavior clearly separated per state.

3) In the context of design patterns, explain the **differences** between: [3 + 3 + 4]

- a) Factor Method vs. Builder
- b) Strategy vs. Template Method.
- c) Proxy vs. Facade.

3) Differences between patterns

3(a) Factory Method vs Builder

Factory Method

- Creates objects by calling a factory method.
- The creation decision (which concrete class) is made by subclasses or by overriding the factory method.
- Best when you want to hide the concrete type and create one object in one step.

Builder

- Builds a complex object step-by-step.
- Useful when the object has many optional parts or configurations.
- Separates construction from representation so you can build different variations using the same building process.

SDA_2025_26_02_Final__Exam

3(b) Strategy vs Template Method

Strategy

- Uses composition: a Context has a Strategy object.
- The algorithm can be changed at runtime by swapping the Strategy implementation.

Template Method

- Uses inheritance: a base class defines a fixed algorithm skeleton.
- Subclasses override specific steps of the algorithm, but the overall structure stays fixed.

SDA_2025_26_02_Final__Exam

3(c) Proxy vs Facade

Proxy

- Has the same interface as the real subject.
- Controls access (validation, security checks, caching, lazy loading, logging etc.).
- Client thinks it is talking to the real object, but proxy sits in between.

Facade

- Provides a simpler interface to a complex subsystem.
- It is not mainly for access control; it is for simplification and hiding internal complexity.

4) In the context of Microservices: [4+3 + 3 points]

a) Explain the **concepts of the Microservice architecture** (including sketch).

b) Which of the following are **benefits of a Microservices Architecture**?
(one to four answers can be correct)

- i. Independent Scaling allows each service to be scaled individually based on its specific load and performance requirements.
- ii. Tightly Coupled Services improve communication efficiency and reduce the complexity of system interactions.
- iii. Polyglot Programming enables different services to use different programming languages, frameworks, and technologies best suited for their functionality.
- iv. Monolithic Deployment ensures easier updates and simplifies version control across all services in the system.

c) Which of the following statements is true regarding **RESTful APIs**?
(one to four answers can be correct)

- i. RESTful APIs store client session data on the server to maintain state across multiple requests.
- ii. RESTful APIs follow a stateless communication model, meaning each request contains all necessary information, and the server does not store client session data.
- iii. RESTful APIs only support JSON as the data format for exchanging information between clients and servers.
- iv. RESTful APIs require all resources to be accessed using only the HTTP GET method.

4) Microservices and REST

4(a) Concepts of Microservices (with sketch description)

Microservices architecture means:

- The application is split into multiple small services.
- Each service focuses on a single business capability (e.g., Orders, Payments, Users).
- Each service can be developed, deployed, and scaled independently.
- Services communicate using APIs or messages.
- Each service often owns its own data store to stay independent.

(A sketch would show multiple services connected via an API gateway or communication layer, each with its own database.)

SDA_2025_26_02_Final__Exam

4(b) Benefits of microservices (correct options)

Correct:

- **i** (Independent scaling of services is a key benefit.)
- **iii** (Different services can use different languages/technologies: polyglot.)

Incorrect:

- **ii** is wrong because microservices aim for loose coupling, not tight coupling.
- **iv** is wrong because microservices do not promote monolithic deployment.

SDA_2025_26_02_Final__Exam

4(c) True statements about RESTful APIs (correct options)

Correct:

- **ii** (REST is stateless: every request contains all required information; server does not store client session state.)

Incorrect:

- **i** is wrong because REST does not store session state on server.
- **iii** is wrong because REST is not limited to JSON (it can use XML, JSON, etc.).
- **iv** is wrong because REST uses multiple HTTP methods (GET/POST/PUT/DELETE...).

5) In the context of **class design principles** [3 + 3 + 4 points]

- a) In the context of the **Interface Segregation Principle (ISP)**, which of the following statements are **true**? (one to four answers can be correct)
- i. The Interface Segregation Principle states that a class should not be forced to implement interfaces it does not use.
 - ii. ISP promotes the creation of large, all-encompassing interfaces to ensure a single point of access for multiple functionalities.
 - iii. ISP encourages splitting interfaces into smaller, more specific ones to prevent clients from depending on methods they do not need.
 - iv. Violating ISP often leads to situations where changes to an interface impact multiple unrelated classes, increasing coupling and reducing maintainability.
- b) In the context of the **Liskov Substitution Principle (LSP)**, which of the following statements are **true**? (one to four answers can be correct)
- i. LSP ensures that a subclass must not weaken or break the contract established by the base class.
 - ii. Violating LSP can lead to runtime errors or unexpected behaviors when substituting a subclass for its base class.
 - iii. A subclass should be able to extend the behavior of a base class without altering its intended functionality.
 - iv. LSP allows subclasses to modify or ignore base class methods as long as they provide additional features.
- c) In the context of the **Open-Closed Principle (OCP)**, which of the following statements is **true**? (one to four answers can be correct)
- i. The Open-Closed Principle states that classes should be open for modification but closed for extension to allow flexibility in design.
 - ii. OCP encourages modifying existing classes frequently to accommodate new requirements, ensuring adaptability.
 - iii. Following OCP means that a class should be open for extension but closed for modification, allowing behavior changes without altering existing code.
 - iv. Violating OCP has no impact on maintainability, as modifying existing code is a natural part of software development.

5) Class design principles (ISP, LSP, OCP)

5(a) ISP — Interface Segregation Principle (correct options)

Correct:

- **i** (Clients should not be forced to depend on methods they do not use.)
- **iii** (Interfaces should be split into smaller, specific interfaces.)
- **iv** (Violating ISP increases coupling and maintenance problems.)

Incorrect:

- **ii** is wrong because large “fat” interfaces are discouraged by ISP.

SDA_2025_26_02_Final__Exam

5(b) LSP — Liskov Substitution Principle (correct options)

Correct:

- **i** (A subtype must be substitutable for its base type without breaking correctness.)
- **ii** (Violations cause bugs/unexpected behavior.)
- **iii** (Subclasses must preserve the meaning/contract of the base class.)

Incorrect:

- **iv** is wrong because ignoring required base behavior breaks substitutability.

SDA_2025_26_02_Final__Exam

5(c) OCP — Open/Closed Principle (correct options)

Correct:

- **i** (Software entities should be open for extension—new behavior added without changing old code.)
- **iii** (Closed for modification—avoid changing existing source/binary when extending behavior.)

Incorrect:

- **ii** is wrong because OCP discourages repeated modification of existing code.
- **iv** is wrong because it contradicts OCP (OCP improves maintainability).

Explanation of every Questions

Question 1:

1) Basic you must know: What is a “pattern” and why Proxy exists

A **design pattern** is a standard solution to a common design problem.

The problem Proxy solves

Sometimes you don't want the client to access a real object directly because you need extra control like:

- validation (check input)
- security (permissions)
- caching
- logging
- lazy loading (create object only when needed)

So you put a **Proxy** in front of the real object.

Proxy idea in one line

Client → Proxy → Real object

Client thinks it is calling the real object, but Proxy is controlling the access.

2) Basic you must know: Roles in Proxy pattern (exam language)

Proxy pattern always has these roles:

1. **Subject (Interface)**
Common interface both Proxy and Real object implement.
2. **RealSubject (Real implementation)**
Does the real work.
3. **Proxy**
Implements same interface, adds extra behavior, forwards calls to RealSubject.
4. **Client**
Uses the Subject interface (doesn't need to know if it's proxy or real).

In your question:

- **Subject** = `FileService` (interface)
 - **RealSubject** = `FileServiceImpl`
 - **Proxy** = `ValidationProxy`
 - **Client** = `Client`
 - **Exception** = `FileNotFoundException` (what happens if file cannot be read)
-

3) Basic you must know: “Same interface” is the key rule

This is the most important rule for Proxy:

- ✓ **Proxy must implement the same interface as the real object.**

Why?

Because the client should be able to use proxy exactly like real service:

- same method name
- same parameters
- same return type

In your exam:

- Interface method: `read(pathname: string): string`
So both:
 - `FileServiceImpl.read(...)`
 - `ValidationProxy.read(...)`
must exist with the same signature.
-

4) Basic you must know: UML class diagram relationships (what to draw)

To draw Q1(a) you need 3 UML relationship ideas:

A) Interface implementation (“implements”)

- A class implements an interface using a line with a triangle arrow (or just state it).
So:
- `FileServiceImpl implements FileService`
- `ValidationProxy implements FileService`

B) Association / “has-a” reference

Proxy must store a reference to the real service:

- `ValidationProxy has realService: FileService (or FileServiceImpl)`

This is how proxy can forward calls.

C) Client depends on interface

Client should use `FileService` not `FileServiceImpl`.

So client is connected to `FileService` (interface), and at runtime it receives proxy.

This is good design because it reduces dependency on concrete class.

5) Basic you must know: What is an object diagram vs sequence diagram

Object diagram (Q1(b))

Object diagram = **real instances at runtime**

It answers: *What objects exist and who points to whom?*

So you draw:

- `clientObj: Client`
 - `proxyObj: ValidationProxy`
 - `implObj: FileServiceImpl`
- And show:
- Client uses proxy
 - Proxy has real impl

Sequence diagram (Q1(c))

Sequence diagram = **time order of messages**

It answers: *Who calls what, step-by-step?*

So you show:

Client → Proxy → (validate) → RealService → return data/throw exception

Now: Teach the exact “thinking” to solve Question 1

Q1(a): How to build the class diagram (step-by-step)

You are given:

- `FileService` (interface)
- `FileServiceImpl` (implements it)
- `FileNotFoundException` (exception)

The question asks: “add ValidationProxy and Client”

So you follow proxy rule:

1. Make `ValidationProxy` implements `FileService`
2. Put a reference inside it: `real: FileService` (or `FileServiceImpl`)
3. Client should call `FileService` (interface), but in diagram connect it to proxy.

✅ That is enough to get full marks.

Q1(b): How to build the object diagram (step-by-step)

Look at runtime objects:

- Client object exists
- Proxy object exists
- Real implementation object exists

Connections:

- Client → Proxy
- Proxy → Real service

That’s the object diagram.

Q1(c): How to build the sequence diagram (step-by-step)

You must show the call flow when doing:

```
Client.read("abc.txt")
```

Correct order:

1. Client calls proxy `read("abc.txt")`
2. Proxy validates pathname
3. Proxy forwards call to real service `read("abc.txt")`
4. Real service returns file content OR throws `FileNotFoundException`
5. Proxy returns content OR propagates exception to client

That is exactly what the exam wants.

Very quick “memory shortcut” for Question 1

When you see the word **Proxy**, immediately think:

✅ “Same interface + holds real object + controls access”

And draw:

- Interface on top (`FileService`)
- Two classes implementing it: (`ValidationProxy`, `FileServiceImpl`)
- Proxy has reference to impl
- Client talks to proxy

Question 2:

1) Basic concept: What is a “state machine”?

A **state machine** describes a system that can be in **one state at a time**, and it changes state when an **event** happens.

Parts of a state machine

- **States:** e.g., TOP, UPWARD, DOWNWARD, BOTTOM
- **Events (triggers):** e.g., `pushUpButton()`, `pushDownButton()`, `endSwitchTop()`, `endSwitchBottom()`
- **Transitions:** arrows from one state to another when an event occurs
- **Actions:** what the system does during a transition or inside a state
 - e.g., motor actions: `runClockwise()`, `runCounterclockwise()`, `stop()`

Simple analogy (elevator/door/window)

A window can be:

- at top (closed)
- moving up
- moving down
- at bottom (open)

Buttons and limit switches cause movement and state changes.

2) Basic problem: Why we don’t want big if/else for states

A beginner implementation often becomes:

- `if(state == TOP && downButton) ...`
- `else if(state == UPWARD && topSwitch) ...`
- `else if(state == DOWNWARD && upButton) ...`

This becomes messy, long, and error-prone.

So we use the **State Pattern**.

3) Basic concept: What is the State Pattern?

State Pattern means:

- ✓ “Put the behavior of each state into its own class.”

Instead of one big class deciding everything, we create:

- One interface **State**
- One class per state (TopState, UpwardState, ...)
- A context class (Controller) that holds the current state object

Key idea

Same event, different behavior depending on state.

Example:

- Pressing Down at TOP starts moving down.
- Pressing Down while already moving down might do nothing.

So each state handles the same event differently.

4) The 3 roles you must know (exam important)

1) Context

The “main” object that the user interacts with.
It holds the current state.

Example name: `Controller`

2) State (interface / abstract class)

Declares the methods for all events.

Example:

`pushUpButton(), pushDownButton(), endSwitchTop(), endSwitchBottom()`

3) Concrete States

One class per state:

- TopState
- UpwardState
- DownwardState
- BottomState

Each implements the event handlers differently.

Now how to answer your Question 2

Q2(a) “Draw class diagram of the State Pattern”

To draw it, follow this checklist:

Step A: Identify all states

From the diagram: TOP, UPWARD, DOWNWARD, BOTTOM
→ you create 4 state classes.

Step B: Identify all events

From the diagram:

`pushUpButton()`, `pushDownButton()`, `endSwitchTop()`, `endSwitchBottom()`

These become methods in the State interface.

Step C: Add Context class

Add `Controller` (or similar) that stores current state:

- `state : State`

and provides public methods for events. Usually:

- Controller has `pushUpButton()` etc.
- It forwards these calls to `state.pushUpButton(this)` (context passed)

Step D: Add Motor to Context

Because actions are about the motor:

- `runClockwise`

- `runCounterclockwise`
- `stop`

So context uses `Motor` (or motor methods inside `Controller`).

Final exam-style description (what must be present)

- `Controller` has a `State`
- `State` interface exists
- Concrete States implement `State`
- `Controller` delegates events to current `State`

That is the answer.

Q2(b) “Steps needed to perform a state transition”

This is always the same logic in State Pattern:

Example transition: from TOP to DOWNWARD when Down button pressed

1. User calls `Controller.pushDownButton()`
2. Controller forwards the event to current state:
 - `state.pushDownButton(controller)`
3. Current state decides what to do:
 - calls motor action (e.g., `motor.runCounterclockwise()`)
4. Current state changes the state in the controller:
 - `controller.setState(new DownwardState())`
5. Done. Now the system behaves as DOWNWARD state.

That sequence is the “steps” they want.

5) What you must remember to score full marks

Always write:

- “Context delegates behavior to state object”
- “Each concrete state handles events”
- “State object changes context’s state when transition occurs”

Mention actions + transitions:

- pressing buttons triggers motor run
 - reaching end switches triggers stop and state update
-

Quick memory cheat

State Pattern is like:

- ✓ “Behavior = depends on current state object.”

Context holds State → state handles events → state can change context state.

Question 3:

Your Q3 asks differences between:

1. **Factory Method vs Builder**
2. **Strategy vs Template Method**
3. **Proxy vs Facade**

SDA_2025_26_02_Final__Exam

0) First basic: Patterns usually answer 3 questions

When you compare two patterns, always ask:

1. **What problem does it solve?**
2. **How does it solve it (mechanism)?**
3. **When should I use it (typical use case)?**

If you can say these 3 for each pattern, you can answer any comparison question.

3(a) Factory Method vs Builder

Factory Method — basic idea

Goal: Create an object without the client knowing the exact concrete class.

Mechanism:

A base “creator” class defines a **factory method**. Subclasses override it to decide what object to create.

When used:

When you want to choose between different product classes (one object per request) and hide that choice.

Tiny example idea

`Dialog.createButton()` returns a `Button` but in different dialogs it returns:

- `WindowsButton` OR
- `LinuxButton`

Client uses `Button` interface; doesn't know which one.

Builder — basic idea

Goal: Build a complex object step-by-step.

Mechanism:

A Builder provides steps like:

- `setPartA()`, `setPartB()`, ...
Then `build()` returns the finished object.

When used:

When construction has many optional parts or order matters:

- Computer (CPU/RAM/GPU)
 - House (rooms, garage, garden)
 - Report (title, charts, sections)
-

Exam difference sentence (easy)

- **Factory Method** = “Which class to instantiate?” (one-step creation)
- **Builder** = “How to assemble a complex object?” (multi-step construction)

What you must know for the exam:

Factory Method is mainly about **polymorphic object creation**; Builder is about **step-by-step construction**.

3(b) Strategy vs Template Method

These two are commonly confused. Here is the core difference.

Strategy — basic idea

Goal: Switch an algorithm/behavior at runtime.

Mechanism: Composition

- Context has a reference to a Strategy interface.
- You can plug in different strategy objects anytime.

When used:

When you want multiple algorithms and want to choose one dynamically:

- payment method (card/paypal)
- sorting algorithm
- compression (zip/rar)
- route planning

Key line:

✅ “Strategy uses composition and can change at runtime.”

Template Method — basic idea

Goal: Keep the skeleton of an algorithm fixed, but allow certain steps to vary.

Mechanism: Inheritance

- Base class defines `templateMethod()` (fixed order).
- Subclasses override some steps.

When used:

When you have a fixed process but some steps differ:

- Report generation skeleton is fixed (load data → format → export), but formatting differs in subclasses.

Key line:

- ✓ “Template Method uses inheritance; skeleton fixed.”
-

Exam difference sentence (easy)

- **Strategy:** swap the whole algorithm by changing the strategy object.
- **Template Method:** algorithm skeleton fixed; subclasses override steps.

Most important exam phrase:

- Strategy = **composition**
 - Template Method = **inheritance**
-

3(c) Proxy vs Facade

These also get confused. One controls access; one simplifies usage.

Proxy — basic idea

Goal: Control access to an object.

Mechanism: Proxy has the **same interface** as the real object.

- It sits in front, then forwards calls if allowed.

Typical reasons:

- validation
- security permission checks
- caching
- lazy loading (create heavy object later)
- logging

Key line:

- ✓ “Proxy = same interface, access control.”

Facade — basic idea

Goal: Provide a simpler interface to a complex subsystem.

Mechanism: Facade wraps **many classes** and exposes a small set of easy methods.

Typical reasons:

- hide complexity
- reduce dependencies
- make subsystem easy for clients

Key line:

✅ “Facade = simplified interface to many classes.”

Exam difference sentence (easy)

- **Proxy:** one object in front of another to control access (same interface).
 - **Facade:** one object in front of many subsystem objects to simplify usage.
-

What basics to memorize to answer Q3 fast

Use this one-page memory

Factory Method

- Creates objects
- one call
- choose concrete class via subclass override

Builder

- constructs complex objects
- step-by-step
- many optional parts

Strategy

- swap algorithm
- composition
- runtime change

Template Method

- skeleton fixed
- inheritance
- override steps

Proxy

- access control
- same interface
- forwards to real object

Facade

- simplify subsystem
- hides many classes
- easy entry point

Question 4:

1) Microservices: the basics you must know

What is Microservices architecture?

Microservices means a big system is split into many **small services**, where each service:

- does **one business capability** (single purpose)
- runs as its own process (often its own deployment)
- can be updated and deployed **independently**
- often owns its own database (so it's independent)

Example services in an online shop:

- User Service
- Order Service
- Payment Service
- Shipping Service

How microservices communicate

They communicate using:

- **APIs** (often HTTP/REST)
- or **messages** (queues, brokers)

They do not work as one big program with direct internal method calls.

What makes microservices different from monolith

- **Monolith:** one big application, one deployment.
 - **Microservices:** many small applications, independent deployment.
-

2) What you should draw/describe in 4(a)

4(a) “Explain concepts of microservices (add sketch)”

To solve 4(a), you should know these keywords (exam loves them):

- ✓ **Service boundaries:** each service has one responsibility
- ✓ **Independent deployment:** services can be released separately
- ✓ **Independent scaling:** scale only the busy service
- ✓ **Loose coupling:** services talk via APIs/messages, not tight dependencies
- ✓ **Decentralized data:** often each service has its own DB

What a correct sketch looks like (in words):

- An API Gateway (optional) in front
 - Multiple services behind it
 - Each service connected to its own database
-

3) Microservices benefits (for 4(b) MCQ)

To answer 4(b), you must know what “benefit” microservices truly provides:

- ✓ **Real benefits**
 - **Independent scaling** (scale Payment Service without scaling everything)
 - **Independent deployment** (update only Order Service)

- **Technology freedom / polyglot** (each service can use different language/framework)
- Better fault isolation (one service crash doesn't kill all)

✗ Not benefits / misunderstandings

- “tightly coupled services” → wrong; microservices aim for **loose coupling**
- “monolithic deployment” → wrong; microservices are the opposite

So, to solve 4(b), you should read each option and ask:

- Does this match microservices' goal of independence and loose coupling?
-

4) REST basics (for 4(c) MCQ)

What is REST?

REST is a style for building web APIs using HTTP.

REST commonly uses:

- resources (like `/users`, `/orders/5`)
- standard HTTP methods:
 - GET (read)
 - POST (create)
 - PUT/PATCH (update)
 - DELETE (delete)

Most important REST rule: Stateless

Stateless means:

- The server does **not** store “session state” for the client between requests.
- Every request contains all the information needed.

Example:

- Client sends request with authentication token every time.
- Server doesn't remember “who you are” from last time (no stored session required).

Data formats

REST is not limited to JSON only:

- JSON is common,
 - but XML, text, etc. are possible too.
-

5) How to answer each part of Q4

Q4(a) What you need to write in exam

Write a paragraph including:

- definition (many small services, single responsibility)
- independent deployment and scaling
- communication via APIs/messages
- each service often owns its data
- plus small sketch description

Q4(b) How to solve the MCQ logically

- Choose options that represent **independence + scaling + technology flexibility**
- Reject options that represent **tight coupling** or **monolithic deployment**

Q4(c) How to solve REST MCQ logically

- Pick statement that matches **statelessness**
- Reject:
 - server-side session storage (that's stateful)
 - “only JSON” (false)
 - “only GET and POST” (false)

Super-short memory cheats (exam gold)

Microservices = “independent small services”

- independent deploy
- independent scale
- loose coupling
- often own DB

REST = “resource + HTTP + stateless”

- stateless requests
- multiple HTTP methods
- many data formats

Question 5:

0) The one basic you must know: What is SOLID?

SOLID is a set of five design principles for writing good object-oriented software. In Question 5 you only need **three** of them:

- **ISP** – Interface Segregation Principle
- **LSP** – Liskov Substitution Principle
- **OCP** – Open/Closed Principle

Each principle answers one design question:

- ISP: How big should interfaces be?
- LSP: Can a subclass safely replace its base class?
- OCP: How should we add new features without breaking old code?

1) ISP — Interface Segregation Principle

What ISP means (in simple words)

✓ “Do not force a class to implement methods it does not use.”

So:

- Prefer **many small, specific interfaces**
- Avoid **one big ‘fat’ interface** with lots of unrelated methods

Why ISP exists

Big interfaces cause problems:

- Classes are forced to implement useless methods.
- Changes in one method affect many classes.
- High coupling and low maintainability.

Simple example

Bad:

- `IMachine { print(), scan(), fax() }`
- A printer class must implement `scan()` and `fax()` even if it can't.

Good:

- `IPrinter { print() }`
- `IScanner { scan() }`
- `IFax { fax() }`

Now each class implements only what it needs.

How to recognize ISP in MCQs (exam logic)

Correct statements usually say:

- “Clients should not be forced to depend on methods they do not use.”
- “Prefer small, focused interfaces.”
- “Violating ISP increases coupling/maintenance problems.”

Wrong statements usually say:

- “One large interface is better.”
 - “It’s fine to include many unrelated methods in one interface.”
-

2) LSP — Liskov Substitution Principle

What LSP means (in simple words)

✅ “If B is a subclass of A, you should be able to use B wherever A is expected without breaking the program.”

So:

- Subclass must behave in a way that **does not violate expectations** of the base class.

Why LSP exists

Polymorphism only works if subclasses are truly substitutable.

If a subclass changes behavior in a surprising way, your program breaks at runtime.

Simple example

Bad design:

- Base class: `Bird` with method `fly()`
- Subclass: `Penguin` extends `Bird` but cannot fly
Now using `Bird` reference with a `Penguin` breaks logic.

Good design:

- Separate `FlyingBird` and `NonFlyingBird`
 - Or don't put `fly()` in the base class
-

How to recognize LSP in MCQs (exam logic)

Correct statements usually say:

- “Subtypes must be substitutable for base types.”
- “Subclass must not break base class contract.”
- “Violating LSP leads to runtime errors or unexpected behavior.”

Wrong statements usually say:

- “Subclass can change behavior arbitrarily.”
 - “It is okay if subclass ignores base class rules.”
-

3) OCP — Open/Closed Principle

What OCP means (in simple words)

✅ “You should be able to add new behavior without changing existing working code.”

This is often summarized as:

Open for extension, closed for modification.

Why OCP exists

- Changing existing code can introduce bugs.
- Extending with new classes is safer than editing old logic.

Simple example

Bad:

- `if(type == "A") doA(); else if(type == "B") doB();`
- To add type C, you must modify the existing if/else block.

Good:

- Use an interface and new classes:
 - Strategy interface
 - AImpl, BImpl, later add CImpl without touching old code.
-

How to recognize OCP in MCQs (exam logic)

Correct statements usually say:

- “Open for extension, closed for modification.”
- “Add new behavior without changing existing code.”

Wrong statements usually say:

- “Modify core code whenever you add new behavior.”
 - “Frequent changes to existing code are expected.”
-

4) How to approach Question 5 in the exam (step-by-step thinking)

When you see MCQs about ISP, LSP, OCP:

1. Identify **which principle** the sub-question is about.
2. Recall the **one-line definition** of that principle.
3. Check each option:
 - Does it match the definition? → correct
 - Does it contradict the definition? → wrong

This prevents confusion even if wording is tricky.

5) Super short memory cheats (write these in your head)

ISP

- “No fat interfaces.”
- “Classes implement only what they use.”

LSP

- “Subclass must be usable wherever base class is used.”
- “No surprising behavior.”

OCP

- “Add new features by extension, not by modifying old code.”